

아쿠아가드(AquaGuard AI) — 재해연쇄·골든타임 대응 에이전트 아키텍처 확정안 v2.4

이 문서는 배경지식이 전혀 없는 3개의 독립된 Claude Code 세션(팀원별 1개씩)이 각자 이 문서 하나만 읽고 자기 트랙을 구현한 뒤, 서로 실행 결과를 한 번도 안 보고도 바로 연동되도록 쓰였다. § 0을 반드시 먼저 읽어라 — “무엇을 만드는지·왜 만드는지”를 이해하지 못한 채 § 5의 스키마만 구현하면 숫자는 맞아도 시스템의 목적을 놓친 코드가 나온다. § 4~§ 5는 “정확히 어떤 형식의 값을 어디서 받아 어떤 형식으로 내보내는지”까지 예시 JSON으로 못박는다. 모호하면 통합 실패 → 실패는 항상 여기(문서)를 먼저 의심할 것. 참고 벤치마크: MOFOM AI(forest-ai-agent, 대상 수상), 자치법규 정책지도(policymaps, 대상 수상).

프로젝트명: **아쿠아가드(AquaGuard AI)** — “물(수해)로부터 지키다”는 의미를 직접 전달하는 이름. 수해(홍수·산사태)를 예방하는 AI라는 정체성이 이름 자체에 드러나도록 확정.

0. 프로젝트 배경 — 왜 이걸 만드는가 (반드시 먼저 읽을 것)

2025년 여름, 한반도는 한 시즌 안에 복합재해를 겪었다. 7월 16~20일 기록적 집중호우(다수 지역 누적 300mm 이상)로 전국에서 37명이 숨지고 12,921명이 대피했다. 피해는 도로침수 778곳, 토사유실 197건, 하천시설 붕괴 403건, 건축물침수 1,857건, 농경지침수 73건, 총 1조 848억원 규모였다. 산사태 하나, 홍수 하나의 문제가 아니라 하나의 집중호우가 사면·하천·도로·건물·농지에 동시에 재해를 일으키는 복합재해(compound hazard) 사건이었다.

가장 심각했던 곳은 경남 산청군이다. 7월 19일 하루 동안 산청군에서 크고작은 산사태가 266건 발생해 13명이 숨지고 1명이 실종됐다. 그런데 이 사건의 진짜 실패 지점은 “예측을 못해서”가 아니다. 산림청은 이미 **7월 17일에** 지자체에 주민 대피를 권고했다. 19일 당일 **오전 8시부터 주민 신고가 빗발쳤음에도**, 산사태 경보가 격상된 시각은 **낮 12시 37분**이었다. 이미 있던 신호가 관(官)의 판단·전파 단계에서 4시간 넘게 새어나간 것이다. 상능마을에서는 일반적인 산사태가 아니라 토양 전체가 서서히 밀려내려오는 ‘땅밀림’ 현상까지 확인됐다 — 사전에 지반 변위를 감지했다면 더 일찍 대응할 여지가 있었다.

한 가지 더, 우연이 아닌 연결고리가 있다. 같은 산청군은 이 산사태로부터 불과 4개월 전인 2025년 3월, 경남·경북 11개 지역(48,238ha)을 태운 대형 산불의 피해지이기도 하다. 산림청 실측 자료에 따르면 산불 피해지는 화재 2년 후까지도 토양 유출량이 일반 산림 대비 3~4배 높게 유지된다 — 나무 뿌리가 토양을 붙잡는 힘을 잃고, 토양 자체가 빗물을 흡수하는 능력을 잃기 때문이다(근거·수치는 § 2.5 참조). 산불이 사면을 훼손해두면, 다음 집중호우가 왔을 때 산사태로 이어질 확률이 크게 높아지는 **재해연쇄(disaster chain)**가 실제로 존재한다.

이 프로젝트가 푸는 문제는 정확히 두 가지다

① **재해는 따로 오지 않고 사슬처럼 이어지는데, 그걸 모델링하는 시스템이 없다.** 산불위험 예측, 산사태 예측, 홍수 예측은 각각 따로 존재한다(예: KIGAM의 물리기반 산사태 모델, § 2.5 참조). 하지만 “산불이 할린 사면 위에 첫 폭우가 떨어지면 산사태 위험이 몇 배로 뛰고, 그 산사태가 하류 하천범람으로 이어질 수 있다”는 인과 사슬을 하나로 연결해 추적하는 시스템은 없다.

② **설령 정확한 예측이 있어도, 그 신호가 실제 대피로 이어지기까지 사람이 판단·전파하는 단계에서 시간이 샌다.** 산청이 정확히 그 사례다 — 예측(산림청 권고)은 있었는데, 그게 실제 경보·대피로 전환되는 데 4시간 넘게 걸렸다. 즉 이 프로젝트의 목표는 “더 정확한 산사태 예측 모델을 만드는 것”이 아니다. 그건 이미 KIGAM 같은 기관이 물리모델로 85~90%대 정확도를 내고 있다. 우리 목표는 “**이미 있는 신호와 실제 대피 사이에서 새는 시간을 자동화로 되찾는 시스템**”을 만드는 것이다.

그래서 아쿠아가드가 하는 일

- 산불 이력(dNBR)이 산사태 위험을 증폭시키는 계수를 문헌·실측 근거로 정량화한다(§ 2.5의 $f(dNBR, \Delta t)$).
- 강수·토양수분·지형·화재이력·지반변위(InSAR)를 실시간으로 감시해 산사태·하천범람 위험을 예측한다(§ 5 Module A/B).
- 위험이 임계치를 넘는 순간, **사람의 판단 단계를 기다리지 않고** 대피소·경로·피해비용까지 자동으로 계산한다(§ 5 Module D/E/G).
- 그 결과를 관(지자체 재난상황실)과 시민에게 **동시에** 전파한다(§ 5 Module O — 골든타임 상태머신).
- 산청 사건의 실제 타임라인(신고 08:00, 경보 12:37)과 이 시스템이 있었다면 나왔을 타임라인을 나란히 비교해, “몇 시간 몇 분을 벌 수 있었는가”를 숫자로 증명한다(§ 9 데모 시나리오).

공모전 트랙 관련

이 프로젝트는 **아이디어/프로토타입 부문**으로 출품하며, 이 부문은 완성도보다 **창의성·독창성**이 최우선 심사 기준이다. 이 문서가 요구하는 수준은 프로덕션급 완성도가 아니라, 위 두 문제의식(①재해연쇄 정량화 ②골든타임 자동화)을 **실제로 동작하는 프로토타입**으로 증명하는 것이다. 데이터 스택(§ 2)·모듈 계약(§ 5)이 자세한 건 신뢰도를 받치는 인프라이기 때문이지, 심사 기준 자체가 완성도는 아니라는 점을 항상 염두에 둘 것 — 데모(§ 9)의 서사가 최종적으로 제일 중요하다.

팀 배경 (왜 이 팀이 이걸 만들 자격이 있는가)

팀원들은 국민대학교 산림환경 및 원격탐사 전공으로, 국가유산청 산불위험 예측 AI 연구(10m 해상도, CNN 기반 멀티태스크 모델, 실제 3개년 연구과제)와 대상 수상작 **MOFOM AI**(forest-ai-agent — Faustmann-Hartman 산림경영 에이전트, 위성 AGB 추정 Quantile RF, FastAPI+Next.js 통합)에서의 실제 협업 경험을 갖고 있다. 이 프로젝트는 그 경험을 화재·산사태 재해연쇄라는 새로운 문제에 적용한 것이며, § 2.5의 화재확터 증폭계수·§ 7의 폴백 계층·§ 4의 모듈화 원칙은 모두 그 선행 프로젝트에서 검증된 패턴을 그대로 재사용한 것이다.

0.1 핵심 컨셉 요약 (TL;DR)

산불로 훼손된 사면 위에 첫 집중호우가 떨어지는 순간부터, 산사태→하천범람→고립까지 이어지는 재해 사슬을 몇 시간 전에 읽어내고, 관(官)의 의사결정을 기다리지 않고 대피소·경로·피해규모까지 자동으로 계산해 동시에 전파하는 에이전트.

- 독창성 축 1 — 재해연쇄(disaster chain):** § 2.5의 $f(dNBR, \Delta t)$ 로 화재→산사태→하천범람 사슬을 정량 모델링(단, 산불 자체는 실시간 예측 대상이 아닌 정적 이력값 — § 0.1 스코프 참조).
- 독창성 축 2 — 골든타임 격차(golden-time gap):** “예측 정확도”가 아니라 “관의 판단·전파 지연”을 자동화로 우회.
- 독창성 축 3 — 판단 지연 자체를 겨냥한 3종 장치(§ 5 Module H, Module O/UI):** ① 시민 신고 역검증 루프(예측 신호를 시민이 즉시 확인해주는 채널) ② 원클릭 승인(사람의 최종 판단권은 유지하되 몇 시간을 몇 초로) ③ What-if 예측 시뮬레이터(가상 강수 시나리오를 심사위원이 직접 조작).
- 스코프 확정:** 이 프로젝트가 실시간으로 예측하는 것은 **수해(산사태·하천범람·도로침수)**뿐이다. 산불은 § 2.5의 정적 증폭계수로만 쓰이고, 별도 화재위험 예측 모델(과거 v2.2의 Module F)은 이번 스코프에서 제외했다 — 데이터 처리 범위가 연중이 아니라 우기 집중기간(6~9월)만으로 좁혔다(§ 2.2).
- 메인 데모 케이스:** 2025년 7월 19일 산청 산사태(§ 0 참조, 공개된 실패 타임라인 보유).
- 보조 케이스:** 서울 취약성지수(기존 소논문 자산) — “전국 확장성 증거” 화면. 메인 서사 아님.
- 트랙:** 아이디어/프로토타입 부문 — 창의성·독창성이 최우선 기준.

1. 재해 Taxonomy

예측 모델은 3종으로 한정한다. 나머지는 GIS 파생 계산으로 흡수한다(행안부 재해연보 분류체계와 1:1 대응).

#	유형	산출 방식	재해연보 대응 카테고리
1	산사태·토사유실	예측 모델 (Module A)	토사유실
2	하천범람·하천시설 붕괴	예측 모델 (Module B)	하천시설 붕괴
3	도로·지하차도 침수	규칙기반 경보 (Module C, 모델 아님)	도로 침수
4	건축물 침수	1·2·3 위험지역 $\cap$ 건축물대장 (파생, Module D)	건축물 침수
5	농경지 침수	1·2·3 위험지역 $\cap$ 농경지 (파생, Module D)	농경지 침수

2. 데이터 스택 (확정)

2.1 정적 레이어 — 10m 그리드

데이터	소스	용도
DEM 유도 지형(경사·곡률·TWI·사면방향)	국토정보플랫폼 5m DEM → 10m 리샘플	Module A 공통입력
NDVI, dNBR	Sentinel-2 (10m)	산불 흉터 매핑, 식생회복 추적
토지피복	환경부 토지피복지도	노출자산 분류

2.2 동적 강제인자 — 500m 그리드, 1시간 단위, 실관측(2019–2025 아카이브 보유)

변수	단위	용도
강수량	mm/h	Module A/B 직접 트리거, 선행누적강우지수(API) 산출 — 필수
VPD	kPa	선택/저우선순위 — 향후 토양수분·증발산 보정(Module A 정밀모드) 확장용, MVP 범위 아님
기온	°C	선택/저우선순위 — "
풍속	m/s	선택/저우선순위 — "

범위 확정 메모: 이 프로젝트는 산불을 별도 예측 대상으로 다루지 않는다(§ 0.1 참조 — 산불은 § 2.5의 $dNBR/f(dNBR, \Delta t)$ 를 통해 “이미 일어난 과거 이벤트가 남긴 정적 증폭계수”로만 쓰인다). 따라서 VPD/기온/풍속처럼 산불위험 실시간 예측에 주로 쓰이던 변수는 이번 스코프에서 필수 입력이 아니다 — 아카이브 확보 자산이라 삭제하지 않고 저우선순위로 남겨둔다.

처리 범위 축소(비용 절감): 우기 집중기간만 처리한다. 500m 관측 아카이브(2019–2025, 연중)를 전부 전처리할 필요는 없다 — 이 프로젝트는 “여름철 집중호우”로 촉발되는 산사태·하천범람·도로침수만 다루므로, **매년 6~9월(우기 집중기간)만 잘라서 처리**하면 데이터 처리량이 연중 대비 약 1/3(4개월/12개월)로 줄어 전처리 시간·저장·컴퓨팅 비용이 그만큼 감소한다. **주의(방법론):** Module A/B가 분류/회귀 학습을 하려면 “위험 발생” 양성 사례뿐 아니라 “비가 왔지만 재해가 안 난” 음성(평시) 사례도 필요한데, 6~9월 구간 안에도 비가 오지 않거나 약한 날이 충분히 섞여 있으므로 — 연중 데이터를 다 쓰지 않아도 음성 사례 부족 문제는 발생하지 않는다. 단, 최종 모델 학습 전 6~9월 구간 내 강우/무강우

비율을 한 번 확인해 클래스 불균형이 심하면 그때 연중 확장을 검토한다(\$ 10 TODO에 반영).

2.3 예보 레이어 — LDAPS, 1.5km, 1일 8회(3시간 간격), 강수량(Rain) 변수 포함 확인됨

- 리드타임 3~6시간 구간만 신뢰. 48시간 전체 신뢰 금지(국지성 집중호우는 6시간 이후 예보력 급락).
- 500m 관측 아카이브와 LDAPS 과거 중첩기간으로 편차보정(bias correction) 함수 학습 → 미래 예보에 적용해 500m급으로 하향 스케일(forest-ai-agent climate_correct 패턴 재사용).
- 편차보정 함수 학습도 \$ 2.2와 동일하게 **6~9월 중첩기간만** 사용하면 충분하다 — 어차피 실사용 시점(경보 대상 기간)도 우기이므로 비우기 데이터로 학습한 보정함수는 실사용 분포와 안 맞을 위험이 오히려 있다.

2.4 SAR/InSAR — Sentinel-1

용도	비고
수체(open water) 탐지	Module B 보조입력, 광학 불가한 호우 상황에서도 동작
InSAR 지반변위 시계열	땅밀림형 산사태 전조. 나지·개활지 등 coherence 확보 지점만 사용 , 산림 피복 사면은 decorrelation으로 신뢰도 낮음을 UI에 명시

2.5 보조 / 화재홍터 증폭계수

- SMAP+SAR 융합 토양수분 (선택 정밀 모드)
- 화재위험 자체는 실시간 예측 대상이 아니다(\$ 0.1) — 아래 f(dNBR, Δt)는 **과거에 이미 난 산불이 남긴 정적(static) 증폭계수**일 뿐, 살아있는 화재위험 모델이 아니다.

f(dNBR, Δt) 근거:

근거	수치	출처
국내 실측(산림청, 2000 동해안 산불)	화재 2년 후에도 토양유출량 일반산림 대비 3~4배	경향신문(2025.4) 인용
침투속도(국제문헌)	화재 6개월 후 강우침투속도 2배 이상	KIGAM
확률증분(그리스 사례)	강우유발 산사태확률 +20~30% 이상	KIGAM
벤치마크	KIGAM 물리모델, 극한강우 후 2.5시간 내 위험도 산출, 예천·경주 85~90%+ 정확도	KIGAM

```
f(dNBR, Δt) = 1 + (A_max(dNBR_class) - 1) × decay(Δt)
A_max: low→1.2, moderate→2.0, high→3.5~4.0 (Key&Benson 2006 등급, 국가유산청 프로젝트와 통일)
decay(Δt): Δt<2년 → 1.0 (감쇠없음) / Δt≥2년 → 자체 NDVI 회복곡선 기반 경험적 산출
```

포지셔닝: KIGAM은 단일 산사태 물리모델 정확도(85~90%)에 집중. 우리는 ①화재이력 증폭(KIGAM에 없음) ②예측 이후 골든타임 자동화까지 통합해 차별화. “KIGAM보다 정확한 모델”이 아니라 “KIGAM급 모델이 있어도 못 막은 전파 지연을 우회하는 시스템”으로 프레임িং. **TODO:** A_max 등급별 수치는 2025.3 산불 11개지역 자체 dNBR 산출 후 사후검증으로 보정.

2.6 벡터 데이터

데이터	소스
대피소 위치	안전Dream / 국가재난안전포털 API
도로망 그래프	국가교통정보센터 표준노드링크
하천 실시간 수위	WAMIS / 한강홍수통제소
건축물	건축물대장
농경지	농림축산식품부 공간데이터
기존 위험지도	산림청 산사태정보시스템, 국토부 지하차도 침수위험지도
피해비용 원단위	행안부 재해연보, 개별공시지가

3. 시스템 아키텍처

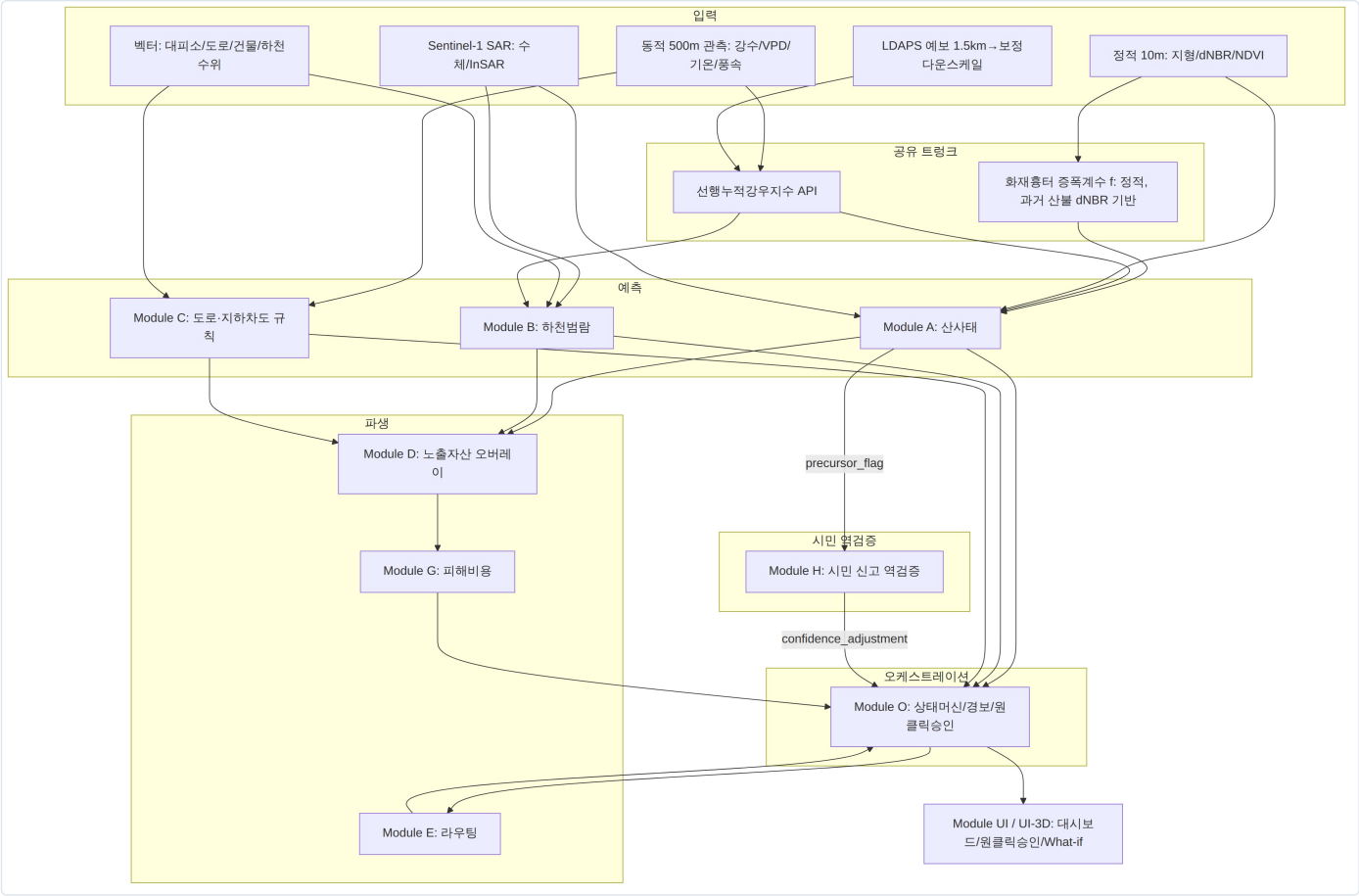


diagram 1

4. 통합 규약 (전 모듈 필수 준수 — 가장 중요한 섹션)

3개의 Claude Code가 서로 대화 없이도 맞아떨어지려면 아래를 토씨 하나 안 틀리고 지켜야 한다. 애매하면 이 섹션이 최우선 근거.

4.1 값 표기 규칙

좌표 필드 네이밍(중요, 전 모듈 공통): 지점은 `x_5179 / y_5179` (미터), 폴리곤·라인은 `geometry_5179` (GeoJSON과 동일한 구조지만 좌표값은 5179 미터 — RFC7946 표준 GeoJSON이 아니므로 `*_geojson`이라 부르지 않는다). Module UI/UI-3D가 최종 출력할 때만 4326으로 재투영해 `*_geojson` 필드명으로 내보낸다. 아래 § 5 예시 중 Module A/E에 이 네이밍을 적용해줬다 — 다른 모듈도 동일 규칙을 따른다.

항목	규칙
좌표계	내부 분석·저장은 EPSG:5179 (한국 UTM-K/중부원점 통합좌표계 — 국내 DEM·KMA 격자·V-World 데이터가 원래 이 계열이라 거리·면적·경사(TWI 등) 계산을 왜곡 없이 할 수 있음). 지형정렬·라우팅·오버레이 등 모든 내부 계산은 5179로 한다. 웹 지도/3D 시각화(deck.gl·MapLibre)로 내보내는 최종 GeoJSON만 EPSG:4326으로 재투영 (pyproj 등 사용, [lon, lat] 순서, RFC 7946 표준). 이 재투영은 Module UI/UI-3D 쪽 출력 직전에만 수행 — 그 앞단(F/A/B/C/D/E/G/O) 모듈 간 교환은 전부 5179 유지
시간	ISO 8601 + 타임존 명시. 예: "2025-07-19T08:00:00+09:00" (KST). UTC 절대 쓰지 않는다
확률	<code>float</code> , 0.0~1.0 (퍼센트 아님)
거리	미터(<code>_m</code> 접미사), 면적은 헥타르(<code>_ha</code>) 또는 제곱미터(<code>_m2</code>) — 필드명에 단위 명시 필수
금액	원(KRW) 정수 (<code>_krw</code> 접미사, 만원 단위 금지)
신뢰구간	항상 [low, high] 2원소 배열, 동일 단위
결측치	<code>null</code> 사용, 빈 문자열/0으로 대체 금지

4.2 모듈 호출 규약

- 모든 모듈은 **Python 패키지**로 구현하고, 표준 진입점 함수 하나로 노출한다: `def run(input: dict) -> dict`
- 각 모듈 폴더는 `pytest`로 **독립 테스트 가능**해야 한다 (다른 모듈 없이도 `tests/`의 fixture만으로 실행 가능).
- 최종 통합은 MOFOM 패턴 그대로: FastAPI 서버 하나(`api_server.py`)가 각 모듈을 `import`해서 순서대로 호출. REST로 쪼개지 않는다(초기 통합 복잡도를 낮추기 위함).

- 모든 모듈 출력은 아래 **공통 봉투(envelope)** 형식을 따른다:

```
{
  "status": "ok",           // "ok" | "degraded" | "error"
  "fallback_tier": 1,       // 1=정상 2=2순위 폴백 3=3순위 폴백
  "data": [ ... ],         // 모듈별 실제 데이터 (아래 §5 참조)
  "warnings": []           // 사람이 읽을 경고 문자열 배열 없으면 빈 배열
}
```

- 모듈은 예외(exception)를 던져서 죽지 않는다.** 실패해도 `status: "degraded"` + 폴백값을 반환해야 오케스트레이터(Module O)가 나머지를 보존할 수 있다 (graceful degradation을 코드 레벨 계약으로 강제).

4.3 계약 파일 공유 규약 — Day 1에 반드시 먼저 할 일

```
contracts/
module_a.schema.json  module_a.example.json
module_b.schema.json  module_b.example.json
module_c.schema.json  module_c.example.json
module_d.schema.json  module_d.example.json
module_e.schema.json  module_e.example.json
module_g.schema.json  module_g.example.json
module_h.schema.json  module_h.example.json
module_o.schema.json  module_o.example.json
```

- 이 문서 §5의 예시 JSON을 그대로 `contracts/module_X.example.json` 으로 저장하고, JSON Schema(타입 검증용)를 `module_X.schema.json` 으로 만들어 셋이 함께 커밋한다.
- 이후 각자 트랙은 **자기 모듈이 소비하는 입력을 `contracts/` 의 `example.json`으로 목업 삼아** 다른 사람 모듈이 완성되기 전에 병행 개발한다.
- `contracts/` 변경은 Day 1 이후 3인 합의 없이 임의 수정 금지 (한쪽에서 필드 하나 바꾸면 다른 두 트랙이 조용히 깨진다).

5. 모듈 계약 — 입출력 예시 JSON (구현 즉시 참조용)

Module A — 산사태 예측 (`module_a_landslide`)

```
// input (x_5179/y_5179는 예시 값 — 실제 값은 pyproj로 4326→5179 변환해 채을 것)
{
  "x_5179": 1090452.3, "y_5179": 1662188.7, "timestamp": "2025-07-19T08:00:00+09:00",
  "static": { "slope_deg": 32.5, "curvature": -0.02, "twi": 6.8, "aspect_deg": 210,
    "dnbr": 0.62, "dnbr_class": "high", "days_since_fire": 121 },
  "dynamic": { "rainfall_1h_mm": [12.0, 18.5, 24.0], "rainfall_cumulative_24h_mm": 187.3,
    "rainfall_cumulative_72h_mm": 245.0, "api_index": 0.81, "source": "observed" },
  "insar_displacement_mm_per_day": null // coherence 미확보 시 null
}
```

`dnbr / dnbr_class / days_since_fire` 는 §2.5의 $f(dNBR, \Delta t)$ 계산에 쓰이는 **정적 값**(과거 산불 이력) — 화재위험을 실시간으로 예측하는 별도 모델은 없다(§0.1 스코프 확정).

```
// output (data 필드) — hours_to_critical 필수: "골든타임"을 실제 숫자로 만드는 필드
{
  "landslide_prob": 0.78, "confidence_interval": [0.65, 0.88], "source": "observed",
  "amplification_factor": 3.1, "precursor_flag": false,
  "hours_to_critical": 2.5,
  "location": { "x_5179": 1090452.3, "y_5179": 1662188.7 } } }
```

hours_to_critical 산출 방법: `landslide_prob` 이 현재값이 아니라 향후 위험임계치(예: 0.7) 초과 시점을 가리키는 값. 관측(과거~현재) 데이터만 있으면 “이미 임계치를 넘었다(0)” 또는 “안 넘었다(null)”만 판단 가능하다. 진짜 “몇 시간 후”를 말하려면 LDAPS 예보(§2.3, 3~6시간 리드타임) 시계열을 1시간 스텝으로 넣어 각 스텝의 `landslide_prob` 을 계산하고, 임계치를 처음 넘는 스텝까지의 시간을 `hours_to_critical` 로 반환한다. 예보 구간을 다 훑어도 안 넘으면 `null` (위험 임박 아님). 폴백: InSAR+실측강수(정밀) → 지형+실측강수 → LDAPS 예보모드(CI 확대, source:"forecast")

Module B — 하천범람 예측 (`module_b_flood`)

```
// input
{
  "reach_id": "GEUMHO_042", "static": { "drainage_area_ko2": 58.2, "river_order": 3, "slope_pct": 1.8 },
  "dynamic": { "rainfall_cumulative_24h_mm": [187.3], "river_level_m": 3.2 },
  "sar_water_extent": null } }
```

```
// output (data 필드) — inundation_extent_5179: 좌표값은 5179 미터 (§4.1 네이밍 규칙), hours_to_critical은 Module A와 동일 산출법
{
  "flood_prob": 0.61, "confidence_interval": [0.45, 0.75], "hours_to_critical": 4.0,
  "inundation_extent_5179": { "type": "FeatureCollection", "features": [ ] } }
```

폴백: 실측강수+수위+SAR → 실측강수+SAR → 실측강수만(CI 확대)

Module C — 도로·지하차도 침수 (`module_c_urban_rule` , 규칙기반·모델 아님)

```
// input
{ "underpass_id": "SC-UP-003", "rainfall_intensity_1h_mm": 45.0
  "known_risk": true, "drainage_capacity_class": "Low" }
```

```
// output (data 필드)
{ "alert_level": "위험", "underpass_id": "SC-UP-003" }
// alert_level ∈ { "경심", "주의", "경계", "위험" }
```

UI 표기 시 신뢰도 배지를 A/B와 다르게(모델 아님을 명시).

Module D — 노출자산 오버레이 (module_d_exposure_overlay)

```
// input — Module A/B/C의 data를 모아 위험폴리곤 배열로 받음 (전부 5179 미터)
{ "risk_polygons": [
  { "source_module": "A", "geometry_5179": { "type": "Polygon", "coordinates": [ ] }, "risk_prob": 0.78 },
  { "source_module": "B", "geometry_5179": { "type": "Polygon", "coordinates": [ ] }, "risk_prob": 0.61 }
],
"building_footprints_5179": { "type": "FeatureCollection", "features": [ ] },
"farmland_parcels_5179": { "type": "FeatureCollection", "features": [ ] } }
```

```
// output (data 필드)
{ "exposed_buildings": [ { "building_id": "B12345", "risk_prob": 0.78, "use_type": "주거" } ],
  "exposed_farmland_ha": 4.2 }
```

Module E — 대피소·경로 라우팅 (module_e_routing)

```
// input (전부 5179 미터) — time_budget_hours는 Module 0가 A/B의 hours_to_critical에서
// 안전여유(경보 인지·준비 시간 등, 기본 30분)를 뺀 뒤 전달
{ "origin": { "x_5179": 1090452.3, "y_5179": 1662188.7 },
  "risk_polygons": [ { "geometry_5179": { }, "risk_prob": 0.78 } ], // Module D 입력과 동일 형식
  "shelter_candidates": [ { "shelter_id": "S001", "x_5179": 1091200.0, "y_5179": 1663500.0, "capacity": 200 } ],
  "road_graph_source": "standard_node_link_v1",
  "time_budget_hours": 2.0 }
```

```
// output (data 필드) — route_5179: 내부 표현. UI 전달 직전에만 route_geojson(4326)으로 재투영
{ "shelter_id": "S001", "route_5179": { "type": "LineString", "coordinates": [ ] },
  "eta_min": 14.5, "route_confidence": "high",
  "time_feasible": true, "time_margin_min": 105.5,
  "fallback_used": false }
```

알고리즘 및 선정 로직: 1. shelter_candidates 각각에 대해 위험가중 A*(위험임계치 risk_prob ≥ 0.7 초과 셀/링크는 그래프에서 제외 또는 가중치 페널티)로 경로·eta_min 계산. 2. eta_min ≤ time_budget_hours×60 을 만족하는(=재해가 닥치기 전에 도착 가능한) 후보들 중 eta_min 이 가장 작은 대피소를 선택. (동률이면 capacity 여유가 큰 쪽 우선 — capacity 기반 부하분산은 § 10 TODO로 확장 가능, MVP는 최단시간 우선으로 충분) 3. **아무 대피소도 시간 안에 도달 불가능하면** (time_feasible: false 가 되는 경우) 원거리 공식 대피소 대신, 위험지역을 벗어나는 가장 가까운 안전 지점(도로망상 risk_prob < 0.3 구간 중 최근접)으로 목적지를 바꾸는 **긴급 폴백**을 실행하고 fallback_used: true 로 표시. UI에는 “지정 대피소까지 시간이 부족합니다 — 가까운 안전지대로 즉시 이동하세요” 경고를 명시적으로 띄운다(생명안전상 가장 중요한 예외처리 — 절대 조용히 넘어가지 않는다).

Module G — 피해비용 추정 (module_g_damage_cost)

```
// input — Module D 출력을 그대로 받음
{ "exposed_buildings": [ { "building_id": "B12345", "risk_prob": 0.78, "use_type": "주거" } ],
  "exposed_farmland_ha": 4.2,
  "unit_cost_table_ref": "재해연보_2024_원단위" }
```

```
// output (data 필드)
{ "estimated_cost_krw": 1250000000, "cost_range_krw": [ 900000000, 1600000000 ],
  "basis_citation": "행안부 재해연보 2024 건축물 침수 원단위" }
```

Module H — 시민 신고 역검증 (module_h_citizen_verification) — 신규(창의성 축 3)

왜 필요한가(§ 0 재상기): 신청 사건의 진짜 실패는 08:00부터 주민 신고가 빗발쳤는데도 관의 경보 격상이 12:37까지 늦어진 것이었다 — 신호는 이미 있었는데 “역으로 검증해 신뢰도를 높이는” 채널이 없었다. Module H는 Module A의 precursor_flag (InSAR 땅밀림 전조)가 뜬 반경 내 주민에게 자동으로 “이상 징후가 보이십니까?”를 푸시하고, 응답을 실시간으로 모아 확신도를 보정해 Module O에 되돌려주는 **역검증 루프**다. 관의 판단을 기다리는 대신, 시민 확인 자체를 자동화된 신뢰도 상승 신호로 쓴다.

```
// input — Module A의 precursor_flag=true 지점 반경 내에서 트리거
{ "alert_id": "AL-20250719-0915", "trigger_location": { "x_5179": 1090452.3, "y_5179": 1662188.7 },
  "trigger_radius_m": 500,
  "citizen_reports": [
    { "report_id": "R001", "x_5179": 1090480.1, "y_5179": 1662201.4,
      "timestamp": "2025-07-19T09:22:00+09:00",
      "report_tie": "이상징후_목격", // "이상징후_동작" | "이미_대피함" | "오팀_신고"
    }
```



```
    "photo_url": null
  }
}
```

```
// output (data 필드)
{ "report_count": 6, "verification_status": "현장확인"
  // "미확인"(응답 0건) | "진장확인" (이상징후 목격 다수) | "오탐관정"(오탐 신고 다수)
  "confidence_adjustment": 0.12 // Module A의 landslide_prob에 가산되는 보정값(-0.3~+0.3) 오탐 다수면 음수
  "response_latency_min": 4.5 }
```

플백: 응답 0건이면 verification_status: "미확인" 으로 두고 confidence_adjustment: 0 — Module O는 시민 확인 없이도 원래 임계치 로직대로 계속 진행한다(역검증은 신뢰도를 “보강”하는 보조 채널이지, 단일 실패점이 아니다).

Module O — 골든타임 오케스트레이션 (module_o_orchestrator)

역할: Module A/B/C를 임계치로 감시 → 초과 시 D/E/G 순차 호출 → Module H로 시민 역검증 트리거 → 경보 패키지 생성 → 전파 상태머신 8단계: 관측 → 예측 → 1차권고 → 시민 역검증(Module H, 병렬) → 원클릭 승인 대기 → 경보격상 → 주민전파 → 개별대피 시간예산 계산: time_budget_hours = A/B의 hours_to_critical - 안전여유(기본 0.5h, 경보 인지·준비 시간) → Module E에 전달 경보 패키지에 Module E의 time_feasible/fallback_used를 반드시 포함 — false인 지역은 UI에서 별도 강조(§ 5 Module E 참조)

원클릭 승인(창의성 축 3-2, 산청의 "4시간 지연" 문제를 정면으로 겨냥):
지자체 담당자가 회의·서면 검토 없이 대시보드에서 버튼 하나로 경보를 승인/보류할 수 있게 한다.
사람의 최종 판단 권한은 유지하되(완전 자동경보 아님), 판단 자체를 몇 시간이 아니라 몇 초로 줄이는 게 목표.
- `approval\_status`: "대기" | "승인" | "거부" | "자동승인(timeout)"
- 엔드포인트: `POST /approve/{alert\_id}` { "decision": "승인"|"거부", "approver_id": "..." }
- **타임아웃 자동승인:** 담당자가 `auto\_approve\_timeout\_min` (기본 15분) 안에 응답하지 않으면 시스템이 자동으로 "승인"으로 전환하고 `approval\_status: "자동승인(timeout)"`으로 기록 — 산청처럼 사람이 늦게 반응해도 경보가 무한정 묶이지 않도록 하는 안전장치. 단, Module H의 `verification\_status`가 "오탐관정"이면 자동승인을 걸지 않고 대기 상태를 유지(오탐 가능성이 높을 때는 사람 판단을 기다림).

```
// output (data 필드)
{ "timeline_actual": { "advisory": "2025-07-17T00:00:00+09:00", "report_start": "2025-07-19T08:00:00+09:00",
  "warning_escalated": "2025-07-19T12:37:00+09:00" },
  "timeline_agent": { "detected": "2025-07-19T09:15:00+09:00", "alert_sent": "2025-07-19T09:20:00+09:00" },
  "golden_time_saved_min": 197,
  "approval_status": "자동승인(timeout)",
  "citizen_verification": { "verification_status": "현장확인", "confidence_adjustment": 0.12 },
  "alert_package": { "landslide": {}, "flood": {}, "shelter_route": {}, "damage_cost": {} } }
```

Module UI / UI-3D

- 대시보드: 지도+타임라인, 골든타임 비교 카운터, 대피소/경로/피해비용 패널, 근거/신뢰도 배지, (보조) 서울 확장성 화면.
- LLM 채팅: 위 모듈 출력(Module O의 alert_package) 위에 얹는 자연어 해석층.
- **원클릭 승인 화면(창의성 축 3-2):** 경보 대기 중인 건마다 “승인 / 거부” 버튼 2개와 근거 요약(위험확률·신뢰구간·시민 역검증 결과·time_feasible)을 한 화면에 압축. 타임아웃 카운트다운을 시각적으로 노출해 “지금 안 누르면 자동승인된다”는 긴장감을 데모에서 직접 보여준다.
- **What-if 예측 시뮬레이터(창의성 축 3-3):** 3D 지도 위에 가상 강수 시나리오 슬라이더(예: “24시간 누적강우량을 150mm~350mm로 바꿔보기”)를 두고, 슬라이더를 움직이면 **새 모델을 만들지 않고** Module A/B의 기존 run() 을 그 가상 강수량으로 즉시 재호출해 산사태·홍수 위험 폴리곤이 실시간으로 다시 그려지게 한다. “만약 산청에 350mm가 왔다면 위험지역이 어디까지 넓어졌을까”를 심사위원이 직접 조작하며 체감하게 하는 것이 목적 — 순수 UI/오케스트레이션 레이어 기능이라 트랙①의 모델 학습과 독립적으로 트랙③에서 구현 가능하다(계약은 § 5 Module A/B의 run(input) 을 그대로 재사용).
- **3D:** MapLibre GL JS(지형) + deck.gl(TerrainLayer/PolygonLayer/PathLayer/ScatterplotLayer). 지형 소스 1순위 V-World 3D/지형 API(시간절약), 2순위 자체 DEM→terrain-RGB. 레이어: ①지형 메시 ②Module A 산사태 위험 드레이프(화재홍터 구간 강조) ③Module B 기반 bathtub 침수볼륨(“정밀 수리모형 아님” 배지 필수) ④건물 압출 ⑤Module E 경로 3D 라인 ⑥시간슬라이더(08:00~12:37, timeline_actual vs timeline_agent 동시 표시) ⑦What-if 강수 슬라이더(위 참조). 스크프: 상능마를 일대 수 km² AOI로 한정.

6. 불확실성 표기 원칙 (전 모듈 공통)

- 모든 확률/추정치는 confidence_interval: [low, high] 와 함께 출력(§ 4.1).
- source: "observed" | "forecast" 를 Module A/B 출력에 필수 포함, forecast는 CI를 더 넓게.
- InSAR는 coherence 확보 지점만 표시, 나머지는 “관측 불가”로 명시.
- Module C는 신뢰도 배지를 A/B와 다르게 표시(모델 아님을 명확히).

7. 플백 계층 (요약)

모듈	1순위	2순위	3순위
A 산사태	InSAR+실측강수 정밀모드	지형+실측강수	LDAPS 예보모드(CI 확대)
B 하천범람	실측강수+하천수위+SAR	실측강수+SAR	실측강수만

H 시민 역검증	시민 응답 다수 확보	응답 소수(낮은 가중치 반영)	응답 0건(confidence_adjustment=0, 원래 로직 유지)
----------	-------------	------------------	--

8. 디렉토리 구조

```
aquaguard-agent/
docs/
  00_index.md
  01_architecture.md      # 본 문서
  DECISIONS.md
contracts/                # § 4.3 — Day 1 최우선 산출물, 3인 공동 커밋
  module_*.schema.json
  module_*.example.json
module_a_landslide/
module_b_flood/
module_c_urban_rule/
module_d_exposure_overlay/
module_e_routing/
module_g_damage_cost/
module_h_citizen_verification/
module_o_orchestrator/
ui/
data/
  static/      # 10m 지형·dNBR·NDVI
  dynamic/     # 500m 관측 아카이브, 6~9월 우기 집중기간만 처리 (§ 2.2)
  vector/      # 대피소·도로·건물·농경지
tests/
scripts/run_all.py
api_server.py      # FastAPI, 전 모듈 import (§ 4.2)
```

9. 데모 시나리오 (산청 2025 타임라인 재연)

- 초기화면: 2025년 3월 산불 피해지(dNBR 흉터) 지도 로딩
- 7/19 강우 시작 → 위험도 그래프가 실제 신고 시작(08:00)보다 앞서 임계치 초과 (백테스트로 목표수치 확정)
- 상능마을 구간 InSAR 이상치 → 땅밀림 전조 플래그
- 대피소·경로·피해비용 패널 동시 생성
- Module B 동시 트리거(하류 영향권)
- 골든타임 비교 카운터: `timeline_actual` (12:37) vs `timeline_agent` → “N시간 O분 확보” 헤드라인
- (보조) 서울 취약성지수 오버레이

10. 남은 TODO

- $f(dNBR, \Delta t)$ 근거 — § 2.5 반영완료. A_max 등급값은 2025.3 산불 11개지역 자체검증으로 보정 필요
- LDAPS↔500m 편차보정 함수 설계·검증(6~9월 중첩기간 기준)
- 산청 타임라인 백테스트로 `golden_time_saved_min` 목표수치 확정
- Module A/B 베이스라인 학습 (2019-2025 아카이브 중 6~9월만 + 산사태정보시스템 라벨). **학습 전 6~9월 구간 내 강우일/무강우일 비율 확인 — 음성 클래스가 부족하면 연중 확장 재검토**(§ 2.2 참조)
- V-World 3D/지형 API 커버리지 확인(상능마을 AOI)
- `contracts/` 실제 파일 생성 (Day 1 최우선, module_h 포함)
- 프로젝트명 확정 — 아쿠아가드(AquaGuard AI)로 확정완료
- Module H 푸시 알림 채널 결정(문자/앱푸시/카톡 알림톡 등 — 데모에서는 웹 대시보드 시뮬레이션으로 대체 가능)
- 원클릭 승인의 `auto_approve_timeout_min` 기본값(15분) 데모 시나리오에 맞춰 조정

11. 팀 구성 및 역할 분담 (3인 체제)

국민대 산림환경 및 원격탐사 전공 3인이 트랙 하나씩 맡는다. 담당자가 바뀌어도 이 표와 § 4~5 계약만 유지되면 재구성 가능.

트랙	담당(제안)	소유 모듈	핵심 산출물
① 예측모델·위성	김민석	A, B	데이터 정렬 파이프라인(6~9월 범위), $f(dNBR, \Delta t)$ 정적 증폭계수 적용, LDAPS 편차보정, 신뢰구간

② 대응로직·데이터통합	나정우	C, D, E, G, H	벡터데이터 수집, 위험가중 A* 라우팅, 피해비용 계산, 시민 신고 역검증 파이프라인
③ 오케스트레이션·UI·3D	하수범	O, UI, UI-3D	상태머신·경보·원클릭승인, Next.js 대시보드, deck.gl+V-World 3D, What-if 시뮬레이터, 산청 데모

(이름은 과거 협업 스킬셋 기준 Claude의 제안 — 실제 배정 시 트랙 성격만 유지하고 이름만 교체하면 됨)

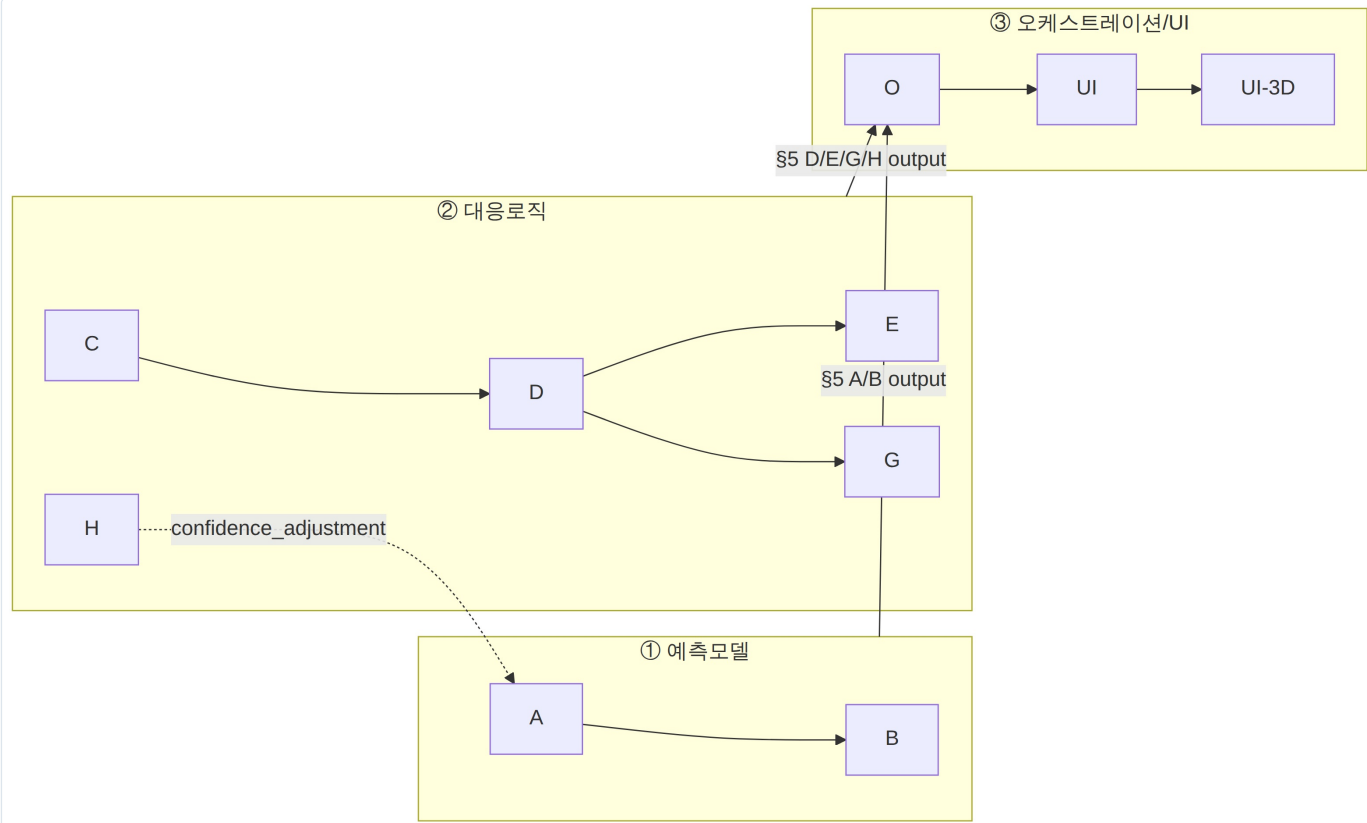


diagram 2

12. 2주 로드맵 (14일)

구간	① 예측모델	② 대응로직	③ 오케스트레이션/UI
Day 1 (계약 고정 — 최우선)	셋이 함께 § 4~5 리뷰, contracts/ 파일 생성·커밋. 이후 각자 트랙 착수	''	''
Day 2-5 (독립 개발)	500m(6~9월만)/1.5km/10m 파이프라인, Module A 베이스라인(목업 InSAR로)	Module C 규칙엔진, Module D(contracts의 A/B example.json을 입력으로 사용), Module H 신고 수집 UI/API 뼈대	Next.js 스캐폴딩 + contracts/module_o.example.json 으로 대시보드 뼈대, V-World API 연동 테스트
Day 6-9 (실데이터 연동)	Module A/B 실데이터 학습·검증, 신뢰구간 산출, 실제 output 공유	실제 A/B output으로 D/E/G 전환 완성, Module H를 Module A의 precursor_flag와 실연동	실제 output 연동, 3D 레이어(위험드레이프·침수볼륨) 구현, 원클릭 승인 화면 프로토타입
Day 10-12 (통합·백테스트)	산청 타임라인 백테스트로 golden_time_saved_min 산출	산청 대피소/경로 실사례 검증	시간슬라이더·What-if 시뮬레이터 완성, api_server.py 통합, 버그픽스
Day 13-14 (마무리)	전원: 발표자료·기획서, 최종 리허설, 예상 질의응답 준비		

13. 트랙별 Claude Code 브리핑 (그대로 복붙해서 각자 세션에 붙여넣을 것)

트랙 ① 예측모델 담당자용

이 저장소의 ARCHITECTURE.md를 읽어라 — § 0(프로젝트 배경)부터 반드시 읽고, 이 시스템이 정확히 "왜" 필요한지(재해연쇄+골든타임 격차, 산청 사건) 이해한 뒤 § 4~5로 넘어가라. 너는 트랙 ①(예측모델·위성) 담당이다.
구현 대상: module_a_landslide, module_b_flood (§ 5 참조). 화재위험을 실시간 예측하는 별도 모델은 없다 — dNBR/f(dNBR,Δt)는 정적 입력값으로만 Module A에 들어간다.
반드시 지킬 것: § 4(통합 규약)의 값 표기 규칙·호출 규약·공통 봉투 형식, § 7 폴백 계층.
다른 트랙(②③)의 내부 구현은 몰라도 된다 — 오직 contracts/ 안의 example.json/schema.json이

네 모듈의 입력이자 출력 계약의 전부다. Day 1엔 나머지 두 세션과 함께 contracts/를 먼저 확정하라.

트랙 ② 대응로직 담당자용

이 저장소의 ARCHITECTURE.md를 읽어라 — § 0(프로젝트 배경)부터 반드시 읽고, 이 시스템이 정확히 "왜" 필요한지(재해연쇄+골든타임 격차, 산청 사건) 이해한 뒤 § 4~5로 넘어가라. 너는 트랙 ②(대응로직·데이터통합) 담당이다.

구현 대상: module_c_urban_rule, module_d_exposure_overlay, module_e_routing, module_g_damage_cost, module_h_citizen_verification (§ 5 참조).

Module D/E/G의 입력은 Module A/B의 실제 구현이 아니라 contracts/module_a.example.json, contracts/module_b.example.json을 목적으로 써서 먼저 개발하라 — 트랙 ①을 기다리지 않는다.

Module H(시민 신고 역검증)는 Module A의 precursor_flag를 트리거로 삼는 독립 모듈이다 — 이것도 contracts/module_a.example.json의 precursor_flag 필드를 목적으로 써서 먼저 개발 가능하다.

반드시 지킬 것: § 4 통합 규약, § 7 폴백 계층. Day 1엔 나머지 두 세션과 함께 contracts/를 먼저 확정하라.

트랙 ③ 오케스트레이션/UI 담당자용

이 저장소의 ARCHITECTURE.md를 읽어라 — § 0(프로젝트 배경)부터 반드시 읽고, 이 시스템이 정확히 "왜" 필요한지(재해연쇄+골든타임 격차, 산청 사건) 이해한 뒤 § 4~5로 넘어가라. 너는 트랙 ③(오케스트레이션·UI·3D) 담당이다.

구현 대상: module_o_orchestrator, ui/(대시보드+채팅+원클릭승인), ui/3d(§ 5 Module UI/UI-3D 참조, What-if 시뮬레이터 포함).

Module O는 A/B/C/D/E/G/H를 전부 contracts/의 example.json으로 mock 호출하는 것부터 시작해 전체 파이프라인을 먼저 굴러보고, 이후 실제 모듈로 하나씩 교체하라.

원클릭 승인 화면은 `POST /approve/{alert\_id}` 엔드포인트와 타임아웃 자동승인 로직(§ 5 Module O)을 반드시 구현하라.

What-if 시뮬레이터는 새 모델이 필요 없다 — Module A/B의 `run()`을 가상 강수값으로 재호출하는 UI 기능이다.

3D 스택은 MapLibre GL JS + deck.gl, 지형은 V-World API 1순위. 데모 AOI는 상능마을 일대로 한정.

반드시 지킬 것: § 4 통합 규약, § 9 데모 시나리오. Day 1엔 나머지 두 세션과 함께 contracts/를 먼저 확정하라.

14. 참고 벤치마크

- USGS — Postfire debris-flow hazards
- 대형 산불 이후 높아진 산사태 우려 - 경향신문(2025.4)
- 산불 뒤 산사태 위험, 재해 예측 기술로 대비한다 - KIGAM
- MOFOM AI / forest-ai-agent
- 자치법규 정책지도 / policymaps
- 2025년 산청 북부 산사태 - 나무위키
- 2025년 여름 한반도 폭우 사태 - 나무위키
- 기상자료개방포털 — LDAPS
- 기상청 API허브
- High-resolution ensemble streamflow predictions using WRF-Hydro and LDAPS (금호강 유역) — <https://jkwra.or.kr/articles/xml/jDww/>